

Pokemon-Datenbankprojekt

Planung, Datenmodell, SQL-Abfragen und Reflexion

Fokus

Relationale Struktur statt
Einzeltabelle

Datenbasis

21 Karten, 29 Attacken-
Verknuepfungen

Ziel

Abfragen erklaren, Model-
entscheidungen begruenden

Team: Robert, Joel, Berkay, Felix, David

Agenda

Teil	Kapitel	Leitfrage
1	Projektplanung	Was war MUSS/SOLL/KANN und welche Entscheidung hielt das Projekt im Zeitrahmen?
2	Datenmodell	Welche Entitäten, Beziehungen, Schlüssel und Normalisierungsideen stecken im Modell?
3	SQL	Welche JOIN-, GROUP-BY- und HAVING-Abfragen zeigen, dass das Modell funktioniert?
4	Qualität und Reflexion	Welche Tests, Lücken und Verbesserungen ergeben sich für eine zweite Version?

Projektplanung: MUSS, SOLL, KANN

MUSS

- Name
- Kartenummer
- Typ
- HP
- Attacken

Ohne diese Felder wäre die Datenbank fachlich nicht vorzeigbar.

SOLL

- Illustrator
- Rarität
- Evolution

Diese Felder machen das Modell interessanter und zeigen echte Beziehungen.

KANN

- Preis
- Schwäche
- Resistenz
- Rückzugskosten

Diese Daten sind sinnvoll, aber nicht entscheidend für den Prototyp.

Projektplanung: wichtigste Entscheidung

Erst Pflichtdaten stabil, dann Erweiterungen.

Warum das geholfen hat

- Die Datenbank war früh abfragbar.
- Fehler im Modell wurden vor der Abgabe sichtbar.
- Zusatzfelder konnten weggelassen werden, ohne das Kernziel zu gefährden.

Konkrete Folge

- 21 Karten haben Name, Nummer, Typ und HP vollständig.
- Attacken sind nicht nur Text, sondern separat modelliert.
- Preis und Resistenz wurden als ehrliche Lücken dokumentiert.

Datenmodellierung: Entitäten

Entität	Was wird gespeichert?	Warum eigene Tabelle?
card	Nummer, Name, HP, Verweise	zentrale Entität, jede Auswertung startet hier
type	Feuer, Wasser, Kampf usw.	Typnamen wiederholen sich bei vielen Karten
rarity	Common, Uncommon, Rare usw.	verhindert Schreibvarianten und Redundanz
illustrator	Name des Illustrators	Namen sollen nicht mehrfach als Freitext stehen
attack	Name, Schaden, Beschreibung	Attacken sind eigene Informationen
card_attack	Karte, Attacke, Reihenfolge, Kosten	technische Umsetzung der mehrwertigen Attackenbeziehung

Warum nicht eine Tabelle? Typen, Raritäten, Illustratoren und Attackentexte würden wiederholt und Korrekturen wären fehleranfällig.

Datenmodellierung: Tabellenübersicht statt Diagramm

Kerntabellen

Tabelle	wichtige Felder
card	id, card_number, name, hp, FK-Felder
attack	id, name, damage, description
card_attack	card_id, attack_id, attack_order, energy_cost

Nachschlagetabellen

Tabelle	Rolle im Modell
type	wiederverwendbare Typnamen
rarity	wiederverwendbare Seltenheitsstufen
illustrator	wiederverwendbare Namen
card auf card	Evolution als Selbstbezug

Kardinalitäten: Beziehungen im Modell

Beziehung	Kardinalität	Bedeutung	technische Umsetzung
Typ zu Karte	1:n	Ein Typ kann bei vielen Karten vorkommen.	<code>card.type_id</code> zu <code>type.id</code> .
Rarität zu Karte	1:n	Eine Seltenheit gilt für viele Karten.	<code>card.rarity_id</code> zu <code>rarity.id</code> .
Karte zu Attacke	n:m	Eine Karte kann mehrere Attacken haben.	<code>card_attack</code> mit <code>card_id</code> und <code>attack_id</code> .
Karte zu Vorentwicklung	Selbstbezug	Eine Karte kann sich aus einer anderen Karte entwickeln.	<code>evolves_from_card_id</code> zu <code>card.id</code> .

Normalisierung: Redundanz vermeiden

Beispiel: Rarität und Typ

- Common, Uncommon, Rare stehen in `rarity`.
- Feuer, Wasser, Kampf stehen in `type`.
- In `card` steht nur die jeweilige ID.

Problem ohne Normalisierung

- Tippfehler wie `Uncommen` würden eigene Gruppen erzeugen.
- Änderungen müssten in vielen Kartenzeilen korrigiert werden.
- Statistikabfragen würden unzuverlässig.
- Attacken als Spalten wären unflexibel bei mehr oder weniger Attacken.

Schlüssel: Primärschlüssel und Fremdschlüssel

Kategorie	bei uns	Funktion
Primärschlüssel	id in card, type, rarity, illustrator, attack	identifiziert jeden Datensatz eindeutig
logische Fremdschlüssel	z.B. card.type_id, card.rarity_id, card.illustrator_id	verbinden Karte mit Nachschlagetabellen
Zwischenschlüssel	card_attack.card_id und card_attack.attack_id	verbinden Karten und Attacken
Selbstbezug	card.evolves_from_card_id	bildet Evolutionsketten innerhalb derselben Tabelle ab

Aktueller Stand: IDs sind vorhanden und inhaltlich konsistent. Echte FOREIGN KEY-Constraints fehlen noch.

Datenqualität: Constraints und Wirkung

Constraint	aktueller Stand	welchen Fehler er verhindern würde
PRIMARY KEY	vorhanden für zentrale Tabellen	doppelte oder nicht eindeutig adressierbare Datensätze
FOREIGN KEY	logisch genutzt, technisch nicht angelegt	Verweise auf nicht existierende Typen, Attacken oder Karten
NOT NULL	kaum konsequent gesetzt	Karten ohne Name, Typ, HP oder Nummer
UNIQUE	nicht konsequent gesetzt	doppelte Kartennummern oder doppelte Typnamen
CHECK	nicht genutzt	unplausible Werte wie negative HP oder leere Energiekosten

SQL-Kompetenz: JOIN-Abfrage

```
SELECT
  c.card_number,
  c.name,
  t.name AS type,
  c.hp,
  i.name AS illustrator,
  r.name AS rarity
FROM card c
JOIN type t ON t.id = c.type_id
JOIN illustrator i ON i.id = c.illustrator_id
JOIN rarity r ON r.id = c.rarity_id
ORDER BY c.id;
```

Schritt für Schritt

- Start in card: hier steht jede Karte.
- type: Join über `type.id = card.type_id`.
- illustrator: Join über `illustrator.id = card.illustrator_id`.
- rarity: Join über `rarity.id = card.rarity_id`.

Warum genau diese JOIN-Bedingungen?

Join-Bedingung	Warum korrekt?	was wäre falsch?
<code>t.id = c.type_id</code>	<code>type.id</code> ist der eindeutige Datensatz, <code>card.type_id</code> speichert den Verweis.	Join über Textnamen wäre anfällig für Schreibweisen.
<code>i.id = c.illustrator_id</code>	trennt Kartendaten und Illustratornamen sauber.	Illustrator direkt in jeder Karte erzeugt Redundanz.
<code>r.id = c.rarity_id</code>	Rarität wird als kontrollierte Kategorie gespeichert.	Freitext-Raritäten würden Statistikfehler erzeugen.

Auswertung: GROUP BY und Aussage

```
SELECT
  t.name AS type,
  COUNT(*) AS cards,
  ROUND(AVG(c.hp), 0) AS avg_hp
FROM card c
JOIN type t ON t.id = c.type_id
GROUP BY t.name
ORDER BY cards DESC, t.name;
```

Ergebnis als Tabelle

Typ	Karten	Ø HP
Feuer	4	165
Wasser	4	95
Kampf	3	77
Pflanze	3	80
Psycho	3	97
Elektro	2	140
Finsternis	2	70

Auswertung: HAVING für Attacken

```
SELECT
  c.name,
  COUNT(*) AS attacks,
  c.hp
FROM card c
JOIN card_attack ca ON ca.card_id = c.id
GROUP BY c.id
HAVING COUNT(*) = 2
ORDER BY c.hp DESC, c.name;
```

Aussage

- 8 von 21 Karten besitzen genau zwei Attacken.
- HAVING filtert nach dem Gruppieren.
- Die Zwischentabelle card_attack macht diese Auswertung einfach.

Ohne Zwischentabelle wäre die Abfrage deutlich unflexibler.

Testen und Debugging

Smoke Tests

- Anzahl der Karten: 21
- Anzahl der Attacken-Verknüpfungen: 29
- Pflichtfelder leer? Nein
- doppelte Kartennummern? Nein

Beispiel-Prüfabfragen

```
SELECT COUNT(*) FROM card;  
SELECT COUNT(*) FROM card_attack;  
SELECT card_number  
FROM card  
GROUP BY card_number  
HAVING COUNT(*) > 1;  
PRAGMA foreign_key_list(card);
```

Ergebnis: Die Inhalte passen logisch zusammen, aber PRAGMA foreign_key_list zeigt keine technisch angelegten Fremdschlüssel.

Datenlücken: Was fehlt für eine bessere DB?

Datenbereich	Stand	Bedeutung
Pflichtfelder	vollständig	Name, Nummer, Typ und HP sind bei allen 21 Karten vorhanden.
Resistenz	1 von 21	fachlich fast leer, deshalb nur vorsichtig auswertbar
Preis	0 von 21	als KANN-Anforderung nicht umgesetzt
Evolutionen	5 echte Links bei 9 Entwicklungskarten	mehrere Vorstufen fehlen in der Auswahl
Constraints	teilweise nur logisch	Datenbank verhindert falsche Eingaben noch nicht selbst

Softwareentwicklung und Reflexion

Was würden wir ändern?

- echte FOREIGN KEY-Constraints im Schema
- NOT NULL, UNIQUE, CHECK konsequent setzen
- card_new entfernen oder sinnvoll integrieren
- Importskript statt manueller Einpflege

Nächster sinnvoller Schritt

- Set-Tabelle mit Setname und Setnummer
- Preis als eigenes Feld oder Verlaufstabelle
- vollständige Evolutionsketten importieren
- kleine Such- oder Filteroberfläche für Karten

Die Grundstruktur funktioniert.

Pflichtdaten sind vollständig, zentrale Beziehungen sind nachvollziehbar modelliert und SQL-Abfragen zeigen die Vorteile der Aufteilung.

Die zweite Version wäre technischer.

Mehr Constraints, automatischer Import, bessere Zusatzdaten und vollständige Evolutionsketten.

Fragen?